

team-alpha-developers

Midterm Evaluation: Alpha Developers (ism6562-midterm-alpha_developers)

Team Members: Harshith Gujjeti, Sana Afroze, Sheikh Robin Emon, Santiago Vela **Repository:** https://github.com/Harshxth/ism6562-midterm-alpha_developers **Level:** Graduate (ISM 6562)
Presentation Topic: SingleStore (MemSQL)

Part 1: Operational Databases, Data Warehousing, and ETL (20 points)

Part 1: Environment Setup (3 pts)

Task	Points	Score	Notes
1.1 Docker compose ps	1	1	Documented
1.2 \dt for all databases	1	1	Present
1.3 pgAdmin screenshot	1	1	Present

Subtotal: 3/3

Part 2: Explore Operational Databases (4 pts)

Task	Points	Score	Notes
2.1 Sales schema + normalization	1.5	1.5	Output and written response present
2.2 HR schema + relationships	1.5	1.5	Present
2.3 Four queries	1	1	All outputs present

Subtotal: 4/4

Part 3: Data Warehouse Concepts (4 pts)

Task	Points	Score	Notes
3.1 Star schema tables	0.5	0.5	All five shown
3.2 Operational vs warehouse comparison	2	2	Present
3.3 Facts vs dimensions	1.5	1.5	Correct identification

Subtotal: 4/4

Part 4: ETL Pipeline (5 pts)

Task	Points	Score	Notes
4.1 ETL script analysis	2	2	Covers all phases
4.2 Count verification	1	1	Present
4.3 New order through ETL	2	2	Flow documented

Subtotal: 5/5

Part 5: Analytical Queries (4 pts)

Task	Points	Score	Notes
5.1 Three analytical queries	3	3	SQL and output present
5.2 EXPLAIN ANALYZE comparison	1	1	Both outputs with comparison

Subtotal: 4/4

Part 1 Total: 20/20

Part 2: Sharding the Sales Database (20 points)

Part 1: Setup & Exploration (4 pts)

Task	Points	Score	Notes
1.1 Docker compose ps	1	1	Documented
1.2 Count rows	1	1	Correct counts
1.3 Uneven distribution analysis	2	2	Hot shard analyzed

Subtotal: 4/4

Part 2: Update ETL Pipeline (8 pts)

Task	Points	Score	Notes
2.1 Diagnose ETL failure	2	2	Correct diagnosis
2.2 Modified ETL script + output	4	4	Working multi-shard ETL; Sana's commit shows proper dual-shard refactoring
2.3 Verify warehouse data + new order	2	2	Verified

Subtotal: 8/8

Part 3: Cross-Shard Challenges (4 pts)

Task	Points	Score	Notes
3.1 Cross-shard query + limitation	2	2	Documented

Task	Points	Score	Notes
3.2 Sharding trade-offs	2	2	Balanced analysis

Subtotal: 4/4

Part 4: EXPLAIN ANALYZE (4 pts)

Task	Points	Score	Notes
4.1 SE shard EXPLAIN	2	2	Present
4.2 Warehouse EXPLAIN + response	2	2	Isolation explained

Subtotal: 4/4

Part 2 Total: 20/20

Part 3: Adding Cassandra for Real-Time Order Ingestion (20 points)

Note: Part 3 has exceptionally detailed task-by-task documentation from Sheikh Robin Emon, with individual .md and .txt files for each task. This is the most granular submission across all teams.

Part 1: Setup & Cassandra Exploration (4 pts)

Task	Points	Score	Notes
1.1 Docker compose ps	1	1	TASK_1_1_VERIFICATION.txt shows all 8 services running
1.2 Nodetool + DESCRIBE	1	1	TASK_1_2_EXPLORATION.txt with full nodetool status and CQL DESCRIBE output
1.3 Table design comparison	2	2	TASK_1_3_COMPARISON.md: thorough analysis of partition-first design, denormalization rationale, single-partition reads

Subtotal: 4/4

Part 2: Query-First Design Analysis (4 pts)

Task	Points	Score	Notes
2.1 Valid Cassandra queries	2	2	TASK_2_1_QUERIES_OUTPUT.txt with both query outputs
2.2 Invalid query + explanation	2	2	TASK_2_2_RESPONSE.md: excellent explanation of full cluster scan problem and partition-key-first design

Subtotal: 4/4

Part 3: Write Orders to Cassandra (4 pts)

Task	Points	Score	Notes
3.1 Dual inserts + verification	2	2	TASK_3_1_INGESTION_OUTPUT.txt: order 28 inserted and verified in both tables
3.2 Dual-write consistency	2	2	TASK_3_2_BATCH_ANALYSIS.md: detailed analysis of logged vs standard batches, batchlog replication

Subtotal: 4/4

Part 4: Update ETL Pipeline (4 pts)

Task	Points	Score	Notes
4.1 ETL analysis	2	2	TASK_4_1_ETL_ANALYSIS.md: identifies import, function, and changes needed
4.2 Modified ETL + verification	2	2	TASK_4_2_WAREHOUSE_VERIFICATION.txt: full Docker build log, 28 orders from Cassandra, warehouse query confirms order 28

Subtotal: 4/4

Part 5: Polyglot Persistence Reflection (4 pts)

Task	Points	Score	Notes
5.1 Comparison table + response	2	2	TASK_5_1_REFLECTION.txt: comprehensive table (PG Shards CA/ACID, Cassandra AP/eventual, Warehouse CP/analytical)
5.2 Cassandra as warehouse argument	2	2	TASK_5_2_ARGUMENT.md: strong 4-point argument against (no JOINS, inefficient GROUP BY, eventual consistency, incompatible with ad-hoc queries)

Subtotal: 4/4

Part 3 Total: 20/20

Part 4: Distributed SQL Database Presentation (40 points)

Database Selected: SingleStore (formerly MemSQL)

Category	Points	Score	Notes
Technical Accuracy	8	_/8	Pending presentation review. Research content (presentation_research_content.md) shows strong understanding: dual storage (rowstore + columnstore), MySQL wire compatibility, hash-based partitioning, 2PC for distributed transactions. Watch for: SingleStore is NOT all-in-memory (tiered storage); no SERIALIZABLE isolation.
TechRetail Evaluation	8	_/8	Research argues SingleStore could replace all three components (HTAP eliminates ETL). Clear trade-off analysis: strong at scale, hard to justify for small workloads due to cost.
Presentation Quality	8	_/8	Pending. PPTX in repo + SharePoint link.

Category	Points	Score	Notes
Visual Aids	6	_/6	Pending
Q&A Handling	2	_/2	Pending
Equal Participation	4	_/4	Roles: Harshith (Part 1 + Part 4), Sana (Part 2), Robin (Part 3), Santiago (Part 4 research)
Slide Content (/10)	—	8	SingleStore (14 slides). Excellent technical accuracy: dual storage (rowstore RAM + columnstore disk), MySQL wire compatibility, hash-based auto-sharding, 2PC, CP positioning all correct. Good attention to limitations (no SERIALIZABLE cluster-wide, no DDL in transactions, no DEFERRABLE constraints). Bold but well-argued TechRetail case for replacing ALL components (rowstore=ingestion+OLTP, columnstore=analytics, zero ETL). Original diagrams: aggregator/leaf node architecture and 2PC commit flow. Comprehensive 10-row comparison table. Cost as drawback noted. Weaknesses: Stock server rack photo on slide 3 adds no value; Spanish template names visible in layout metadata; 14 slides slightly high for timing.
GitHub Repository	4	4	Well-organized repo with parts 01-04. All 4 members have commits (Robin: 14, Harshith/Harshxth: 10, Sana: 2, Santiago: 4). Prof-tcsmith has access. PPTX and detailed research markdown in part04/. Part 3 has exceptional task-level documentation (9 individual files).

Part 4 Total (GitHub only): 4/4

Part 4 Total (with presentation): _/40 — Fill in from in-class rubric

GitHub Repository Assessment

Criterion	Status
Repo organized by parts	Yes -- part01/ through part04/ with full Docker infrastructure
Prof-tcsmith invited	Yes
All members have commits	Yes -- all 4 members
Part 4 materials in repo	Yes -- PPTX, SharePoint link, detailed research markdown
Submission docs present	Yes -- .docx for all parts
Modified ETL scripts	Yes -- parts 01-03 etl/ directories
Task documentation	Exceptional -- Part 3 has 9 individual task files (TASK_.md, TASK_.txt)

Summary

Component	Points Available	Score
Part 1	20	20
Part 2	20	20
Part 3	20	20
Part 4 (GitHub only)	4	4

Component	Points Available	Score
Part 4 (Presentation)	36	_/36
Total (excluding presentation)	64	64

Strengths

- **Part 3 documentation is exemplary** -- Sheikh Robin Emon's task-by-task files with actual command outputs, written analyses, and verification evidence set the standard for what thorough documentation looks like. This level of granularity makes grading straightforward and demonstrates genuine understanding.
- Strong division of labor: each member owned a part and delivered completely
- SingleStore research content is detailed and well-structured with clear use-case analysis
- Robin's commit history (14 commits for Part 3 alone) shows methodical, incremental work
- Good .gitignore practices (excluding Word temp files, DS_Store, pycache)

Areas for Improvement

- Sana Afroze has only 2 commits -- more incremental contributions would be beneficial
- Research markdown filename has a typo ("contect" instead of "context")
- Part 1 Dockerfile already includes cassandra-driver (not needed until Part 3) -- minor but shows forward-copying without cleanup